

Raport z listy 8

Modele regresji i ich zastosowania

Ksawery Józefowski, album 277513

2026-05-06

Spis treści

1	Wstęp	1
2	Zadanie 1	2
3	Zadanie 2	2
4	Zadanie 3	3
5	Zadanie 4	5
6	Zadanie 5	5
7	Zadanie 6	6
8	Zadanie 7	6
9	Zadanie 8	7
10	Zadanie 9	7

1 Wstęp

Rozważamy model regresji nieliniowej

$$Y = g(\mathbf{x}, \beta) + \varepsilon.$$

Niech

$$n = 10, \quad (\beta_1, \beta_2, \beta_3, \sigma) = (10, 3, 2, 1/2) \quad x_i = \frac{i}{2} \quad \text{dla} \quad i = 1, 2, \dots, n.$$

2 Zadanie 1

Generujemy n obserwacji $y_1, \dots, y_n$ postaci

$$y_i = \frac{\beta_1 x_i}{\beta_2 + x_i} + \beta_3 + \varepsilon_i, \quad i = 1, 2, \dots, n,$$

gdzie $\varepsilon_1, \dots, \varepsilon_n$ są niezależnymi zmiennymi losowymi o rozkładzie $\mathcal{N}(0, \sigma^2)$.

```
set.seed(123)
n <- 10
x <- (1:n) / 2
beta.1 <- 10
beta.2 <- 3
beta.3 <- 2
sigma.kwadrat <- (1/2)^2
varepsilon <- rnorm(n, mean = 0, sd = sqrt(sigma.kwadrat))
y <- (beta.1 * x) / (beta.2 + x) + beta.3 + varepsilon
```

3 Zadanie 2

Funkcja g jest postaci

$$g(x, \beta) = \frac{\beta_1 x}{\beta_2 + x} + \beta_3$$

Jej gradient G jest postaci

$$G(\mathbf{x}, \beta) = \nabla g(\mathbf{x}, \beta) = \begin{bmatrix} \frac{\partial g}{\partial \beta_1} \\ \frac{\partial g}{\partial \beta_2} \\ \frac{\partial g}{\partial \beta_3} \end{bmatrix} = \begin{bmatrix} \frac{x}{\beta_2 + x} \\ -\frac{\beta_1 x}{(\beta_2 + x)^2} \\ 1 \end{bmatrix}$$

Implementujemy te wzory w funkcjach `g()` oraz `gradient()`.

```
g <- function(x, beta.1, beta.2, beta.3) {
  return((beta.1 * x) / (beta.2 + x) + beta.3)
}

gradient <- function(x, beta.1, beta.2, beta.3) {
  g.1 <- x / (beta.2 + x)
  g.2 <- -(beta.1 * x) / (beta.2 + x)^2
  g.3 <- rep(1, length(x))
  return(c(g.1, g.2, g.3))
}
```

4 Zadanie 3

Przyjmujemy następujące oznaczenia:

- $X = [x_{i,j}] \in \mathcal{M}(\mathbb{R}^{n \times p})$ – macierz wartości realizacji zmiennych objaśniających,
- $Y = [y_1, y_2, \dots, y_n]^T$ – wartości realizacji zmiennej objaśnianej,
- $\beta^{(0)} \in \mathbb{R}^p$ – początkowa wartość wektora β ,
- $g(X; \beta) = [g(x_1; \beta), g(x_2; \beta), \dots, g(x_n; \beta)]^T$ – wartości dla ustalonej funkcji g w oparciu o zadane parametry β i wartości realizacji zmiennych objaśniających,
- $G^{(j)} = \left[\frac{\partial g(x_i; \beta)}{\partial \beta_j} \right]_{i,j} \in \mathcal{M}(\mathbb{R}^{n \times p})$ – macierz gradientów funkcji g ,
- $L(\beta; \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2 - \sum_{i=1}^n \frac{(y_i - g(\cdot; \beta))^2}{2\sigma^2}$ – logarytm funkcji wiarygodności,
- $S(\beta) = \|Y - g(X; \beta)\|^2$ – kwadrat normy wektorów Y i $g(X; \beta)$.

Kolejne kroki algorytmu są następujące:

- wybieramy punkt startowy parametrów $\beta^{(0)} \in \mathbb{R}^p$ oraz liczby $\varepsilon > 0$ i $M \in \mathbb{N}_+$,
- tworzymy ciąg iteracji:

$$\beta^{(j+1)} = \beta^{(j)} + [(G^{(j)})^T G^{(j)}]^{-1} (G^{(j)})^T [Y - g(X; \beta^{(j)})],$$

- algorytm zatrzymujemy w kroku m , jeśli

$$\|\beta^{(m)} - \beta^{(m-1)}\| \leq \varepsilon \quad \text{lub} \quad m = M,$$

gdzie zgodnie z treścią zadania kładziemy

$$\varepsilon = 0.0001, \quad M = 100.$$

- przyjmujemy wówczas, że $\hat{\beta} = \beta^{(m)}$ oraz $\hat{\sigma}^2 = \hat{\sigma}^2(\hat{\beta})$.

W przypadku gdy macierz $(G^{(j)})^T G^{(j)}$ jest nieodwracalna, liczymy uogólnioną macierz odwrotną np. za pomocą funkcji `ginv()` z pakietu **MASS**.

Implementujemy zatem opisany wyżej algorytm.

Rozpoczynamy od wzorów na $G^{(j)}$ w funkcji `G.j()`.

```
M <- 100
epsilon <- 0.0001

G.j <- function(x, beta.1, beta.2, beta.3) {
  g.1 <- x / (beta.2 + x)
```

```

g.2 <- -(beta.1 * x) / (beta.2 + x)^2
g.3 <- rep(1, length(x))
return(cbind(g.1, g.2, g.3))
}

```

Następnie implementujemy wzory na ciąg iteracji $\beta^{(j)}$ w funkcji *beta.j()*.

```

beta.j <- function(x, y, beta.1, beta.2, beta.3) {
  G <- G.j(x, beta.1, beta.2, beta.3)
  g_vec <- g(x, beta.1, beta.2, beta.3)
  residuals <- y - g_vec
  GTG <- t(G) %*% G
  delta <- ginv(GTG) %*% t(G) %*% residuals
  beta_new <- c(beta.1, beta.2, beta.3) + as.vector(delta)
  return(beta_new)
}

```

Pozostaje już zaimplementować sam algorytm Gaussa-Newtona w funkcji *Gauss.Newton()*.

```

Gauss.Newton <- function(x, y, beta.1, beta.2, beta.3, M, epsilon) {
  beta_curr <- c(beta.1, beta.2, beta.3)
  n <- length(x)

  result <- matrix(NA, nrow = M + 1, ncol = 4)

  # Krok startowy
  residuals_start <- y - g(x, beta_curr[1], beta_curr[2], beta_curr[3])
  sigma2_start <- sum(residuals_start^2) / n
  result[1, ] <- c(beta_curr, sigma2_start)

  for (j in 1:M) {
    beta_new <- beta.j(x, y, beta_curr[1], beta_curr[2], beta_curr[3])

    residuals <- y - g(x, beta_new[1], beta_new[2], beta_new[3])
    sigma2 <- sum(residuals^2) / n

    result[j + 1, ] <- c(beta_new, sigma2)
    if (norm(beta_new - beta_curr, type = "2") <= epsilon) {
      result <- result[1:(j + 1), , drop = FALSE]
      break
    }
  }

  beta_curr <- beta_new
}

```

```

    return(result)
}

```

oraz funkcję wiarygodności opisaną powyżej.

```

wiarogodnosc <- function(x, y, beta.1, beta.2, beta.3, sigma) {
  n <- length(x)
  residuals <- y - g(x, beta.1, beta.2, beta.3)
  L <- -n/2 * log(2 * pi) - n/2 * log(sigma^2) - sum(residuals^2) / (2 * sigma^2)
  return(L)
}

```

5 Zadanie 4

kroki algorytmu	$\hat{\beta}_1^{(s)}$	$\hat{\beta}_2^{(s)}$	$\hat{\beta}_3^{(s)}$	$\hat{\sigma}^2^{(s)}$	$L\left(\hat{\beta}^{(s)}, \hat{\sigma}^2^{(s)}\right)$
1	8.00000	2.00000	1.00000	1.6173782	-16.5934174
2	8.72025	0.01766	0.03557	7.0016935	-23.9201456
3	8.39791	0.33692	-0.28738	0.7071344	-12.4567126
4	3.49340	1.14187	5.83770	3.7793900	-20.8371984
5	9.82105	0.40243	-0.41809	1.8914785	-17.3761794
6	6.19687	0.98491	3.17199	1.3160947	-15.5627293
7	9.89279	0.86380	-0.44462	0.1394239	-4.3382021
8	9.86612	0.92197	-0.40077	0.1291566	-3.9557361
9	9.89111	0.91657	-0.43202	0.1291433	-3.9552217
10	9.88984	0.91714	-0.43002	0.1291433	-3.9552210
11	9.88999	0.91708	-0.43024	0.1291433	-3.9552210
12 (ostatni)	9.88997	0.91709	-0.43021	0.1291433	-3.9552210

Wnioski: Algorytm zatrzymał się po 12 krokach, spełniając przyjęte kryterium stopu. Otrzymane estymatory odbiegają od prawdziwych wartości parametrów, co sugeruje zbieżność do lokalnego, a nie globalnego minimum funkcji sumy kwadratów residuów. Spowodowane to jest prawdopodobnie źle dobranym punktem startowym.

6 Zadanie 5

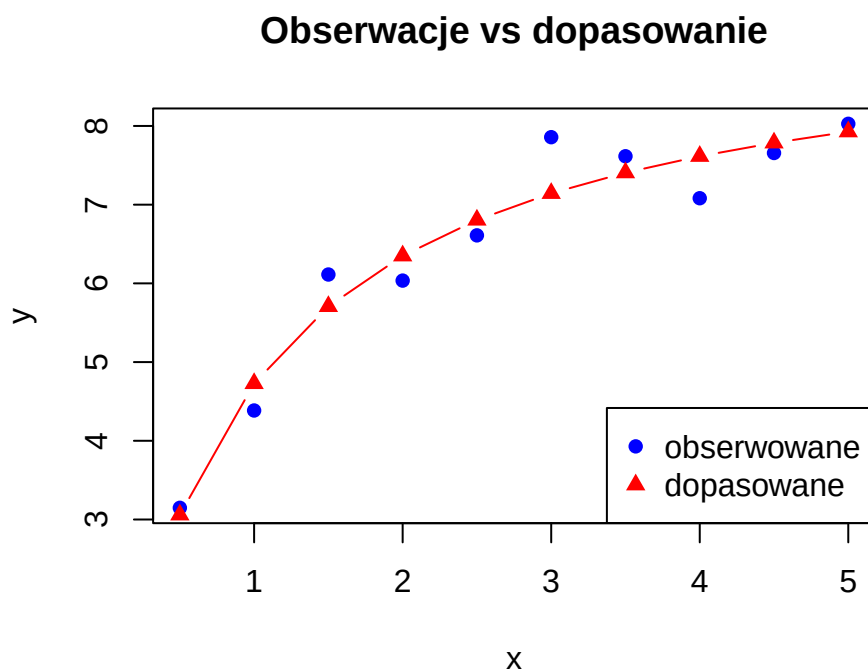
parametr	prawdziwe	estymowane
beta1	10.00	9.88997
beta2	3.00	0.91709
beta3	2.00	-0.43021

Wnioski: Estymatory parametrów β_2 i β_3 znacznie odbiegają od prawdziwych wartości, natomiast β_1 jest stosunkowo dobrze oszacowany

7 Zadanie 6

```
y.hat <- g(x, beta.hat[1], beta.hat[2], beta.hat[3])

plot(x, y, pch = 16, col = "blue",
     xlab = "x", ylab = "y",
     main = "Obserwacje vs dopasowanie")
lines(x, y.hat, pch = 17, col = "red", type = "b")
legend("bottomright",
     legend = c("obserwowane", "dopasowane"),
     col = c("blue", "red"),
     pch = c(16, 17))
```



Wnioski: Na wykresie widać, że krzywa dopasowana przebiega blisko punktów obserwowanych, jednak ze względu na błędne estymatory parametrów dopasowanie nie oddaje w pełni prawdziwej zależności między zmiennymi.

8 Zadanie 7

Wobec wzoru na macierz kowariancji estymatora najmniejszych kwadratów w modelu regresji liniowej, przyjmujemy że (macierz G pełni rolę macierzy X z modelu regresji)

$$Cov(\hat{\beta}) = \sigma^2(G^T G)^{-1}$$

Aby otrzymać estymator $Cov(\hat{\beta})$ zastępujemy nieznany wektor β (w G) wektorem $\hat{\beta}$, a wariancję σ^2 szacujemy za pomocą błędu średniokwadratowego

$$s^2 = \frac{\sum_{i=1}^n [y_i - g(x_i, \hat{\beta})]^2}{n - p}.$$

```
kowariancja <- function(s.kwadrat, G) {
  return(s.kwadrat * ginv(t(G) %*% G))
}
```

	1	2	3
1	2.81464	-0.82230	-3.63704
2	-0.82230	0.33989	1.26593
3	-3.63704	1.26593	5.13321

Wnioski: Oszacowana macierz kowariancji estymatora $\hat{\beta}$ wskazuje na stosunkowo duże wariancje poszczególnych estymatorów oraz silne współliniowości między nimi, co potwierdza niepoprawność otrzymanego rozwiązania.

9 Zadanie 8

```
tab.2 <- Gauss.Newton(x, y, beta.1 = 9.7, beta.2 = 2.9, beta.3 = 2,
  M = M, epsilon = epsilon)

tab.3 <- Gauss.Newton(x, y, beta.1 = 1, beta.2 = 1, beta.3 = 1,
  M = M, epsilon = epsilon)
```

10 Zadanie 9

punkt.startowy	beta1.hat	beta2.hat	beta3.hat	sigma2.hat
(8, 2, 1)	9.88997	0.91709	-0.43021	0.12914
(9.7, 2.9, 2)	9.88997	0.91709	-0.43022	0.12914
(1, 1, 1)	0.22156	-1.07878	6.45993	1.68221

Wnioski: Dla punktów startowych (8, 2, 1) oraz (9.7, 2.9, 2) algorytm zbiegł do tego samego lokalnego minimum, natomiast dla punktu startowego (1, 1, 1) otrzymano zupełnie inne rozwiązanie z wyraźnie większą wariancją residuów. Wynika z tego, że algorytm Gaussa-Newtona jest wrażliwy na wybór punktu startowego i nie gwarantuje znalezienia globalnego minimum.